Applied Cryptography
Week 5: Hash Functions and Keyed Hashing

Bernardo Portela

M:ERSI, M:SI, M:CC - 26

What is a Hash Function?

Hash functions are everywhere

- Key derivation
- Digest for authentication
- Randomness extraction
- Password protection
- Proofs of work

What is a Hash Function?

Hash functions are everywhere

- Key derivation
- Digest for authentication
- Randomness extraction
- Password protection
- Proofs of work

Not only in crypto:

- Indexing in version management
- Deduplication in cloud storage systems
- File integrity in intrusion detection



Describing Hash Functions

The hash output is short, aka hash, fingerprint or digest

Cryptographic hash functions give strong security guarantees

Use hash as an identifier

- Cryptographic hash functions cannot be injective ($f(a) = f(b) \Rightarrow a = b$)

Describing Hash Functions

The hash output is short, aka hash, fingerprint or digest

Cryptographic hash functions give strong security guarantees

Use hash as an identifier

- Cryptographic hash functions cannot be injective ($f(a) = f(b) \Rightarrow a = b$)
  - **Why?**

Describing Hash Functions

The hash output is short, aka hash, fingerprint or digest

Cryptographic hash functions give strong security guarantees

Use hash as an identifier

- Cryptographic hash functions cannot be injective ($f(a) = f(b) \Rightarrow a = b$)
  - **Why?**
- Yet they should be *well distributed* and *unpredictable*
- Hash values can identify arbitrarily large inputs

Describing Hash Functions

The hash output is short, aka hash, fingerprint or digest

Cryptographic hash functions give strong security guarantees

Use hash as an identifier

- Cryptographic hash functions cannot be injective ($f(a) = f(b) \Rightarrow a = b$)
  - **Why?**
- Yet they should be *well distributed* and *unpredictable*
- Hash values can identify arbitrarily large inputs

Signing $H(m)$ should be **as secure** as signing m

Hash functions need to be deterministic and public

- Everyone should be able to recompute hash/identifier
- ... So what do we mean by security here?

Secure Cryptographic Hash Functions

Efficient algorithms with nice properties

- Unpredictable outputs
- Hard to find pre-images
- Hard to find collisions

Secure Cryptographic Hash Functions

Efficient algorithms with nice properties

- Unpredictable outputs
- Hard to find pre-images
- Hard to find collisions

Hash functions are validated heuristically

- Similar to process for AES
- International competition to select designs
- Competitors are scrutinized wrt security and performance
- Several rounds, so more eyes on small number of proposals
- Most recent one: SHA-3

#1: Pre-image resistance

It is *hard* to find the input that produced a given hash value

How can we establish this in concrete terms?

#1: Pre-image resistance

It is *hard* to find the input that produced a given hash value

How can we establish this in concrete terms?

Pre-image experiment

- Let $\mathcal{S}$ be the set of pre-images (domain)
- Let $\mathcal{R}$ be the set of images (counterdomain)
- Do $x \xleftarrow{\$} \mathcal{S}$; $y \leftarrow h(x)$
- Attacker is given value y
- Attacker outputs $x' \in \mathrm{S}$ and wins if $h(x') = y$

#2: Collision Resistance (CR)

- By definition, collisions **must exist**.
  - Recall that $|\mathcal{S}| \gg |\mathcal{R}|$
- This can be argued from the *pigeonhole principle*
  - If you have m holes and n pigeons to put in these holes, if $n > m$, at least one hole will have more than one pigeon!
- But can we find $m_0 \neq m_1$ s.t. $h(m_0) = h(m_1)$?

#2: Collision Resistance (CR)

- By definition, collisions **must exist**.
  - Recall that $|\mathcal{S}| \gg |\mathcal{R}|$
- This can be argued from the *pigeonhole principle*
  - If you have m holes and n pigeons to put in these holes, if $n > m$, at least one hole will have more than one pigeon!
- But can we find $m_0 \neq m_1$ s.t. $h(m_0) = h(m_1)$?

Think of the best possible hash function.

Q1: What could that be?

#2: Collision Resistance (CR)

- By definition, collisions **must exist**.
  - Recall that $|\mathcal{S}| \gg |\mathcal{R}|$
- This can be argued from the *pigeonhole principle*
  - If you have m holes and n pigeons to put in these holes, if $n > m$, at least one hole will have more than one pigeon!
- But can we find $m_0 \neq m_1$ s.t. $h(m_0) = h(m_1)$?

Think of the best possible hash function.

Q1: What could that be?

Q2: Is finding collisions harder or easier than finding pre-images?

Breaking Hash Functions

Attack that finds a pre-image

- Search through all possible pre-images (brute-force)
- Consider a perfect hash function with output of n bits
- Cost: about 2^n operations
- Absolutely infeasible for modern hash functions
  - $n = 256$ for SHA-256 and BLAKE2s

Breaking Hash Functions

Attack that finds a pre-image

- Search through all possible pre-images (brute-force)
- Consider a perfect hash function with output of n bits
- Cost: about 2^n operations
- Absolutely infeasible for modern hash functions
  - $n = 256$ for SHA-256 and BLAKE2s

And if we want to find another pre-image (break second pre-image resistance)?

- Nothing better than before
- Keep trying different values until you guess correctly

Breaking Hash Functions

Attack that finds a pre-image

- Search through all possible pre-images (brute-force)
- Consider a perfect hash function with output of n bits
- Cost: about 2^n operations
- Absolutely infeasible for modern hash functions
  - $n = 256$ for SHA-256 and BLAKE2s

And if we want to find another pre-image (break second pre-image resistance)?

- Nothing better than before
- Keep trying different values until you guess correctly

But what if we only want to find a collision?

Finding Collisions

Collisions can be found with work $\sqrt{2^n}$, much better than 2^n!

Methodology

- Compute values like the brute-force attack
- Store them in a data structure *indexed by image value*
- Each new image value is searched in data structure
- Repeat until a collision is found

Finding Collisions

Collisions can be found with work $\sqrt{2^n}$, much better than 2^n!

Methodology

- Compute values like the brute-force attack
- Store them in a data structure *indexed by image value*
- Each new image value is searched in data structure
- Repeat until a collision is found

How many operations?

- After q values, we checked $q * (q-1)/2$ pairs **Q: why?**

Finding Collisions

Collisions can be found with work $\sqrt{2^n}$, much better than 2^n!

Methodology

- Compute values like the brute-force attack
- Store them in a data structure *indexed by image value*
- Each new image value is searched in data structure
- Repeat until a collision is found

How many operations?

- After q values, we checked $q * (q-1)/2$ pairs **Q: why?**
- Checking 2^q pairs takes roughly $\sqrt{2^q}$ values
- Overall complexity is that of finding the pre-image of a hash with $n/2$ bits of output (half the bits!)

The birthday paradox (not very paradoxical, just counterintuitive)

Implication of Birthday Attacks

For CR, hash outputs must be 2x security parameter

- 128-bit security $\rightarrow$ 256-bit hashes
- 256-bit security $\rightarrow$ 512-bit hashes

Implication of Birthday Attacks

For CR, hash outputs must be 2x security parameter

- 128-bit security $\rightarrow$ 256-bit hashes
- 256-bit security $\rightarrow$ 512-bit hashes

We can use security-parameter-sized hash outputs when:

- Security against arbitrary collisions is not required
- E.g. we might only need pre-image resistance
- Deriving a key from a secret input



Key Takeaways



- Hash functions are one-way functions
  - From any sized inputs to fixed-size output
  - Never injective



Key Takeaways



- Hash functions are one-way functions
  - From any sized inputs to fixed-size output
  - Never injective
- Easy to go from x to $f(x)$
- Hard to go from $f(x)$ to x



Key Takeaways



- Hash functions are one-way functions
  - From any sized inputs to fixed-size output
  - Never injective
- Easy to go from x to $f(x)$
- Hard to go from $f(x)$ to x
- Pre-image resistance
  - Given y, give me any x for which $f(x) = y$
- Collision resistance
  - Give me any x_1, x_2 for which $f(x_1) = f(x_2)$
  - Much easier!!



Key Takeaways



- Hash functions are one-way functions
  - From any sized inputs to fixed-size output
  - Never injective
- Easy to go from x to $f(x)$
- Hard to go from $f(x)$ to x
- Pre-image resistance
  - Given y, give me any x for which $f(x) = y$
- Collision resistance
  - Give me any x_1, x_2 for which $f(x_1) = f(x_2)$
  - Much easier!!
- For output of 2^n, collision can be found in $\approx 2^{\frac{n}{2}}$
- So for 2^{128} resistance, output must be at least 2^{256}
- Birthday attack

Building Hash Functions

Two main approaches that use iterative processes

- **Merkle-Damgård construction:** Used for MD4, MD5, SHA-1, SHA-256, SHA-512. Relies on a $m + n$-to-n bits compression function to construct a hash function of output length n for arbitrary input lengths

Building Hash Functions

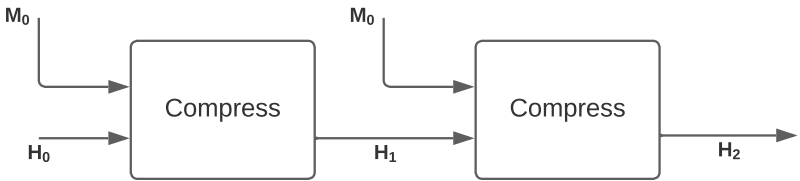
Two main approaches that use iterative processes

- **Merkle-Damgård construction:** Used for MD4, MD5, SHA-1, SHA-256, SHA-512. Relies on a $m + n$-to-n bits compression function to construct a hash function of output length n for arbitrary input lengths
- **Sponge construction:** Used for SHA-3, uses a l-bit permutation to construct a hash function for arbitrary input and output lengths

Merkle-Damgård Construction

All prominent hash functions from 80s-2000s.

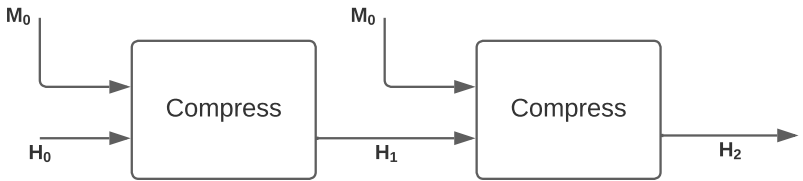
- H_0 is the initial value: constant and **public**
- M is broken into blocks of size m, $M_1, M_2, \ldots$



Merkle-Damgård Construction

All prominent hash functions from 80s-2000s.

- H_0 is the initial value: constant and **public**
- M is broken into blocks of size m, $M_1, M_2, \ldots$



- SHA-256: block size 512, output size 256 bits
- SHA-512: block size 1024, output size 512 bits
- What if messages are not of the same size as the block?

Merkle-Damgård Construction – Padding

Padding is always added to the message

- Append a 1 bit to the message
- Fill with zeros up to 64/128 bits away from the block end
- Last 64/128 bits encode the message length in bits

Merkle-Damgård Construction – Padding

Padding is always added to the message

- Append a 1 bit to the message
- Fill with zeros up to 64/128 bits away from the block end
- Last 64/128 bits encode the message length in bits

E.g. we want to hash the 8-bit string 10101010 using SHA-256

Message is: 10101010**1**00000(...)000001000

Merkle-Damgård Construction – Padding

Padding is always added to the message

- Append a 1 bit to the message
- Fill with zeros up to 64/128 bits away from the block end
- Last 64/128 bits encode the message length in bits

E.g. we want to hash the 8-bit string 10101010 using SHA-256

Message is: 10101010**1**00000(...)000001000

Q: Can't we just pad by adding 0s?

Merkle-Damgård Construction – Security

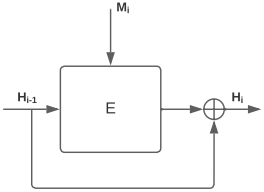
Useful result

- If the compression result is CR (for small inputs), then..
- ... the whole construction is CR (for arbitrary inputs)

To break the hash function you must break the compression function

Compression Functions: Davies-Meyer

All popular MD constructions use the Davies-Meyer construction:

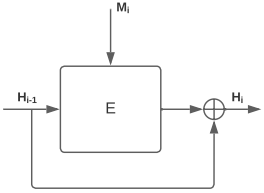


Block ciphers used as compression functions!

- Message is the encryption key!
- Construction creates a fixed point when $H_{i-1} = D(M_i, 0)$

Compression Functions: Davies-Meyer

All popular MD constructions use the Davies-Meyer construction:



Block ciphers used as compression functions!

- Message is the encryption key!
- Construction creates a fixed point when $H_{i-1} = D(M_i, 0)$

$$H_i = E(M_i, H_{i-1}) \oplus H_{i-1}$$
$$H_i = E(M_i, D(M_i, 0)) \oplus D(M_i, 0)$$
$$H_i = H_{i-1}$$

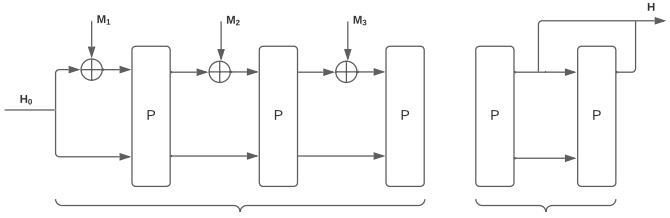
Sponge Construction

A more recent alternative to MD is the sponge construction

It relies on a fixed (non-keyed) permutation

Very Versatile

- Varying input/output lengths
- PRGs and stream ciphers
- PRFs and keyed hashes



Sponge Construction – Description

The state is the same size w as the permutation input

Sponge Construction – Description

The state is the same size w as the permutation input

Absorb

- Fixed initial value h_0, gradually accumulate message into state
- Message broken into blocks of size r (rate)
- Block is smaller than state size
- Block XOR'ed into state
- Permutation recomputed

Sponge Construction – Description

The state is the same size w as the permutation input

Absorb

- Fixed initial value h_0, gradually accumulate message into state
- Message broken into blocks of size r (rate)
- Block is smaller than state size
- Block XOR'ed into state
- Permutation recomputed

Squeeze

- Dual process iteratively constructs output
- Output constructed block by block
- Permutation computed over the entire state
- Block-sized part of the state is accumulated in the output

MD5

- Broken! 128-bit output
- Most popular hash function until broken in 2005
- These days, it takes seconds to find collisions
- The SHA function family (next) uses a similar design

Secure Hash Algorithm (SHA)

Standardized by NIST in the US. International *de facto* standard

SHA-0 published in '93

- Both with 160-bit outputs
- Vulnerability not public at the time
- Later discovered collision attack in $2^{60} << 2^{80}$ operations
- More recent attacks reduced it to 2^{33}

Secure Hash Algorithm (SHA)

Standardized by NIST in the US. International *de facto* standard

SHA-0 published in '93

- Both with 160-bit outputs
- Vulnerability not public at the time
- Later discovered collision attack in $2^{60} << 2^{80}$ operations
- More recent attacks reduced it to 2^{33}

Replaced with SHA-1 in '95, which remained unbroken until quite recently – (**2017**)

Most applications currently use SHA-2 (256 or 512 bits)

- Same design principles; larger parameters

Newer applications adopt SHA-3 (Sponge)

- Flexible output size is very useful!

SHA-1 Internals

- Merkle-Damgård, with Davies-Meyer compression function
- Block cipher used in compression function called SHACAL-1
  - Block cipher with 160-bit block sizes!
  - ARX – Add, Rotate, XOR

SHA-1 Internals

- Merkle-Damgård, with Davies-Meyer compression function
- Block cipher used in compression function called SHACAL-1
  - Block cipher with 160-bit block sizes!
  - ARX – Add, Rotate, XOR
- Message blocks are 512-bits, hashes are 160-bits long
- Davies-Meyer addition (not XOR): five 32-bit additions
- Insecure! Expected collisions in 2^{63} ops in 2005
- SHAttered in 2017

SHA-1 Internals

- Merkle-Damgård, with Davies-Meyer compression function
- Block cipher used in compression function called SHACAL-1
  - Block cipher with 160-bit block sizes!
  - ARX – Add, Rotate, XOR
- Message blocks are 512-bits, hashes are 160-bits long
- Davies-Meyer addition (not XOR): five 32-bit additions
- Insecure! Expected collisions in 2^{63} ops in 2005
- SHAttered in 2017

```
SHA1-blockcipher(a, b, c, d, e, M) {
  W = expand(M);
  for i = 0 to 79 { // K are constants
    new = (a <<< 5) + f(i, b, c, d) + e + K[i] + W[i]
    (a, b, c, d, e) = (new, a, b >>> 2, c, d)
  }
  return (a, b, c, d, e)
}
```

SHA-2 Family

- Family of 6 hash functions
  - 32 Bit: SHA-224; SHA-256
  - 64-Bit: SHA-384; SHA-512; SHA-512/224; SHA-512/256

SHA-2 Family

- Family of 6 hash functions
  - 32 Bit: SHA-224; SHA-256
  - 64-Bit: SHA-384; SHA-512; SHA-512/224; SHA-512/256
- Three digit identifier defines the output length
- Increased parameters and improved internal block ciphers
- SHA-224 and 256 still use 512 bit blocks (64 rounds)
  - SHA-224 is exactly the same as SHA-256, but has different IV and truncated output
  - SHA-384 and SHA-512 are similarly related
- SHA-512 compression function very similar, but has 80 rounds

SHA-2 Family

- Family of 6 hash functions
  - 32 Bit: SHA-224; SHA-256
  - 64-Bit: SHA-384; SHA-512; SHA-512/224; SHA-512/256
- Three digit identifier defines the output length
- Increased parameters and improved internal block ciphers
- SHA-224 and 256 still use 512 bit blocks (64 rounds)
  - SHA-224 is exactly the same as SHA-256, but has different IV and truncated output
  - SHA-384 and SHA-512 are similarly related
- SHA-512 compression function very similar, but has 80 rounds

No attack on full SHA-2 beats generic attacks

- Still SHA-3 was (prudently) developed with different design
- Also has the benefit of varying sized outputs
- Good to generate keys!

SHA-3

- Keccak selected in 2012
- 5-year NIST SHA-3 competition
- Competition called for new design, if SHA-2 gets attacked

SHA-3

- Keccak selected in 2012
- 5-year NIST SHA-3 competition
- Competition called for new design, if SHA-2 gets attacked

Keccak is very different and very flexible

- Sponge based with 1600-bits permutation (in SHA-3)
- Blocks can be 1152, 1088, 832 or 576 bits
- Corresponding to 224, 256, 384 or 512 bit outputs
- As a bonus we get the SHAKE functions
  - SHAKE128 and SHAKE256
  - eXtendable Output Functions (XOFs)
  - You can specify output length



Key Takeaways



- Two main constructions: MD and Sponge



Key Takeaways



- Two main constructions: MD and Sponge
- Merkle-Damgård
  - Used in MD5; SHA-1; SHA-2
  - Sequential compression of message – DM
  - Davies-Meyer compresses using a block cipher



Key Takeaways



- Two main constructions: MD and Sponge
- Merkle-Damgård
  - Used in MD5; SHA-1; SHA-2
  - Sequential compression of message – DM
  - Davies-Meyer compresses using a block cipher
- Sponge
  - Used in SHA-3 and SHAKE
  - Absorb phase permutes message
  - Squeeze phase retrieves the output



Key Takeaways



- Two main constructions: MD and Sponge
- Merkle-Damgård
  - Used in MD5; SHA-1; SHA-2
  - Sequential compression of message – DM
  - Davies-Meyer compresses using a block cipher
- Sponge
  - Used in SHA-3 and SHAKE
  - Absorb phase permutes message
  - Squeeze phase retrieves the output
- SHA-2 and SHA-3 currently the *de facto* standards

MACs as Keyed Hashes

Short Summaries of Potentially Large Messages

- Called a hash if everything is public
- Keyed hashes allow for conditional hash computation

MACs as Keyed Hashes

Short Summaries of Potentially Large Messages

- Called a hash if everything is public
- Keyed hashes allow for conditional hash computation

Message Authentication Codes – MACs

- Symmetric Authentication $t \leftarrow \mathrm{MAC}(k, m)$
- t guarantees that m was produced by someone that knows k
- Implies message m was not changed since its creation
- Digital signatures in the symmetric paradigm!

Message Authentication Codes

Typical use of MACs – SSH, IPSec, TLS

- Two parties want message authentication and integrity
- Some form of set-up/agreement to establish common key k
- Sender computes $t \leftarrow \mathrm{MAC}(k, m)$ and sends (m, t)
- Receiver gets (m, t), recomputes $t' \leftarrow \mathrm{MAC}(k, m)$
- If $t \neq t'$, message is rejected!

Message Authentication Codes

Typical use of MACs – SSH, IPSec, TLS

- Two parties want message authentication and integrity
- Some form of set-up/agreement to establish common key k
- Sender computes $t \leftarrow \mathrm{MAC}(k, m)$ and sends (m, t)
- Receiver gets (m, t), recomputes $t' \leftarrow \mathrm{MAC}(k, m)$
- If $t \neq t'$, message is rejected!

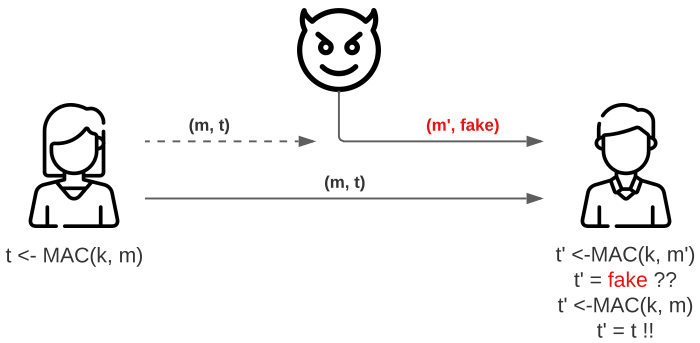
Acceptance means m was produced while knowing k

In this process, message is public!

MACs do not give confidentiality. They provide integrity

It's orthogonal to encryption. In real-world applications, we will need to combine these

Authentication and Message Integrity



- No possibility of computing t without k implies
- Adversary cannot change the message
- Adversary cannot conjure new messages

MAC Security

Standard notion is UF-CMA

- Goal: Unforgeability
- Adversary power: Chosen Message Attacks

MAC Security

Standard notion is UF-CMA

- Goal: Unforgeability
- Adversary power: Chosen Message Attacks

Security Experiment

- Experiment generates a key k
- Adversary (adaptively) sends m to get $t \leftarrow \mathrm{MAC}(k, m)$
- Eventually, attacker outputs (m^*, t^*)

Attacker wins if $t^* = \mathrm{MAC}(k, m^*)$, and if m^* was not queries in the experiment.
Contrary to IND-CPA, a victory here implies a broken MAC scheme.

MAC Security Nuances

- MAC on its own does not protect against replay attacks
- Suppose a network scenario
  - Attacker sees authenticated message (m, t)
  - Delivers (m, t) multiple times
  - MAC will verify every time!

MAC Security Nuances

- MAC on its own does not protect against replay attacks
- Suppose a network scenario
  - Attacker sees authenticated message (m, t)
  - Delivers (m, t) multiple times
  - MAC will verify every time!
- Simple technique: impose message never repeats in network
- Sequence numbers or timestamps
  - Prepend counter and keep counter as state in both sides
  - Prepend timestamp (local clock reading)
  - How should the receiver operate in both cases?

Some Context

MACs constructed from hash functions and block ciphers

Simplest construction: prefix key

$$\mathrm{MAC}(k, m) = \mathrm{H}(K||M) \text{ or } \mathrm{PRF}(k, m) = \mathrm{H}(K||M)$$

Some Context

MACs constructed from hash functions and block ciphers

Simplest construction: prefix key

$$\mathrm{MAC}(k, m) = \mathrm{H}(K||M) \text{ or } \mathrm{PRF}(k, m) = \mathrm{H}(K||M)$$

Q: Wait. Why not $\mathrm{H}(M||K)$**?**

Some Context

MACs constructed from hash functions and block ciphers

Simplest construction: prefix key

$$\mathrm{MAC}(k, m) = \mathrm{H}(K||M) \text{ or } \mathrm{PRF}(k, m) = \mathrm{H}(K||M)$$

Q: Wait. Why not $\mathrm{H}(M||K)$**?**

MD yields insecure MAC and PRF!

- Given (m, t), attacker outputs $\mathrm{H}(k||m||pad||m')$
- This can be computed just from t and m'
- Adversary must know $|k| + |m|$ (which often is not hard)
- **Length extension attack**

Some Context

MACs constructed from hash functions and block ciphers

Simplest construction: prefix key

$$\mathrm{MAC}(k, m) = \mathrm{H}(K||M) \text{ or } \mathrm{PRF}(k, m) = \mathrm{H}(K||M)$$

Q: Wait. Why not $\mathrm{H}(M||K)$**?**

MD yields insecure MAC and PRF!

- Given (m, t), attacker outputs $\mathrm{H}(k||m||pad||m')$
- This can be computed just from t and m'
- Adversary must know $|k| + |m|$ (which often is not hard)
- **Length extension attack**

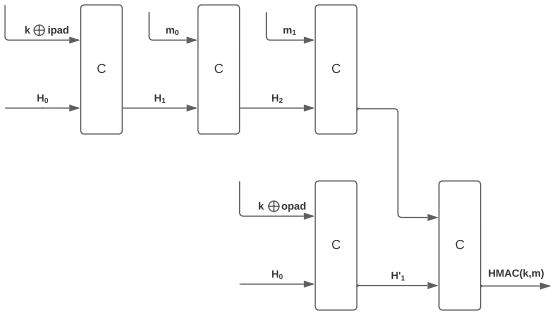
A consideration in SHA-3 construction

- Abandon MD construction

HMAC Construction

When instantiated with MD construction

- Compression function is PRF $\rightarrow$ Secure MAC
- HMAC is simply $\mathrm{H}((K \oplus opad)||\mathrm{H}((k \oplus ipad)||m))$
- *ipad* and *opad* are constants: align to block size



On Collision-based Forgeries

Hash function collisions $\rightarrow$ hash-based MAC forgeries

However, attacker cannot easily search for them w/o key

On Collision-based Forgeries

Hash function collisions $\rightarrow$ hash-based MAC forgeries

However, attacker cannot easily search for them w/o key

Collisions in MAC can also yield forgeries

- Collisions occur when $\sqrt{2^n}$ MACs are issued

Building MACs from Block Ciphers

We have seen block ciphers $\rightarrow$ hash functions $\rightarrow$ MACs

But there are also direct constructions: block ciphers $\rightarrow$ MACs

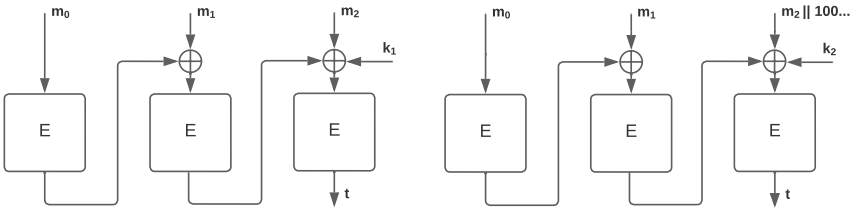
CMAC

- Used in IPSec
- CMAC improves CBC-MAC (broken for variable-length m)
- Use CBC mode of operation
- Fix IV to the all-zero block
- Take the last ciphertext block as a tag

CMAC Internals

CMAC fixes CBC-MAC by processing last block differently

- All blocks except last are processed like CBC-MAC
- Keys k_1 and k_2 derived from k
  - $l \leftarrow E(k, 0)$
  - $k_1 = (l << 1) \oplus (\texttt{0x00..0087} * MSB(l))$
  - $k_2 = (k_1 << 1) \oplus (\texttt{0x00..0087} * MSB(k_1))$



Custom MAC Constructions

More efficient MAC constructions are designed from scratch

Poly1305 is one such construction by D. J. Bernstein

Based on

- Universal Hash Functions
- Wegman-Carter construction

Universal Hash Functions

UHF are a weak form of Hashing

- Don't need to be collision resistant
- Parametrized by a key $\mathsf{UH}(k, m)$
- Guarantee that, for two fixed messages $m_0 \neq m_1$:

$$\Pr[\mathsf{UH}(k, m_0) = \mathsf{UH}(k, m_1)] \leq \epsilon$$

- Considering random k and very small ϵ

Universal Hash Functions

UHF are a weak form of Hashing

- Don't need to be collision resistant
- Parametrized by a key $\mathsf{UH}(k, m)$
- Guarantee that, for two fixed messages $m_0 \neq m_1$:

$$\Pr[\mathsf{UH}(k, m_0) = \mathsf{UH}(k, m_1)] \leq \epsilon$$

- Considering random k and very small ϵ

No other security requirements $\rightarrow$ easy to construct

We can use a universal hash function as a MAC

Provided that we only authenticate **one message!**

Wegman-Carter Construction

How to circumvent this limitation?

- Use a PRF to strengthen the UH
- Converts a UH into a fully secure MAC
- AES can fill the PRF role!

Wegman-Carter Construction

How to circumvent this limitation?

- Use a PRF to strengthen the UH
- Converts a UH into a fully secure MAC
- AES can fill the PRF role!

Intuition: Encrypt Universal Hash Value

$$\mathrm{UH}(k_1, m) \oplus \mathrm{PRF}(k_2, n)$$

- The full MAC key is (k_1, k_2)
- n is a public value that must never repeat
  - A.k.a. a nonce
- This can be kept as a counter, or generated at random

Poly1305-AES: Wegman-Carter in Practice

- Initial proposal used AES as the Wegman-Carter PRF
- The universal hash function uses prime $p = 2^{130} - 5$

$$\text{Poly1305}((k_1, k_2), m) = ((m_1 k_1 + \ldots + m_i k_1^i \bmod p) + \text{AES}(k_2, n)) \bmod 2^{128}$$

- Blocks are 128 bits and last block is padded with 100
- All blocks set bit 129, so MSB is 1
- The final addition is performed modulo 2^{128}
- TLS pairs Poly1305 with ChaCha20, not with AES



Key Takeaways



- Keyed hashing allows for message authentication
- For hash function h, one cannot produce $h(k, x)$ w/o k
- Provides integrity; ciphers give confidentiality



Key Takeaways



- Keyed hashing allows for message authentication
- For hash function h, one cannot produce $h(k, x)$ w/o k
- Provides integrity; ciphers give confidentiality
- Just add a key as prefix!
  - Problem! Length extension attacks with MD hashes



Key Takeaways



- Keyed hashing allows for message authentication
- For hash function h, one cannot produce $h(k, x)$ w/o k
- Provides integrity; ciphers give confidentiality
- Just add a key as prefix!
  - Problem! Length extension attacks with MD hashes
- HMAC
  - Inner and outer keys $k \oplus ipad$, $k \oplus opad$



Key Takeaways



- Keyed hashing allows for message authentication
- For hash function h, one cannot produce $h(k, x)$ w/o k
- Provides integrity; ciphers give confidentiality
- Just add a key as prefix!
  - Problem! Length extension attacks with MD hashes
- HMAC
  - Inner and outer keys $k \oplus ipad$, $k \oplus opad$
- CMAC
  - Do AES-CBC with an all-zero IV; return the last block
  - With a twist to prevent extension attacks



Key Takeaways



- Keyed hashing allows for message authentication
- For hash function h, one cannot produce $h(k, x)$ w/o k
- Provides integrity; ciphers give confidentiality
- Just add a key as prefix!
  - Problem! Length extension attacks with MD hashes
- HMAC
  - Inner and outer keys $k \oplus ipad$, $k \oplus opad$
- CMAC
  - Do AES-CBC with an all-zero IV; return the last block
  - With a twist to prevent extension attacks
- Wegman-Carter
  - Use a UHF for a unique message
  - XOR it with an encryption of a nonce
  - Used in AES-GCM (next class)

Applied Cryptography
Week 5: Hash Functions and Keyed Hashing

Bernardo Portela

M:ERSI, M:SI, M:CC - 26